Experiment 07

SVM – Linear and Non-Linear Kernels

```
# Experiment 07: SVM Linear and Non-Linear Kernels

# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

# 2. Load Wine dataset
data = load_wine()

X = data.data
y = data.target

# 3. Convert dataset into DataFrame
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y

# 4. Display first five records
print("First Five Records:")
print(df.head())

# 5. Check dataset information
print("\nDataset Information:")
df.info()

# 6. Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Check class distribution
print("\nClass Distribution:")
print(df['target'].value_counts())

# 7. Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y
)

print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

# 8. Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 9, 10, 11. Create SVM models
models = {
    'Linear': SVC(kernel='linear'),
    'Polynomial': SVC(kernel='poly', degree=3),
    'RBF': SVC(kernel='rbf')
}

# 12. Train and predict each model
results = []

for name, model in models.items():

    print("\n==========", name, "Kernel ==========")

    # Train model
    model.fit(X_train, y_train)

    # Predict output
    y_pred = model.predict(X_test)

    # 13, 14. Evaluation metrics
    accuracy = accuracy_score(y_test, y_pred)
```

```
    precision = precision_score(
        y_test, y_pred, average='weighted'
    )

    recall = recall_score(
        y_test, y_pred, average='weighted'
    )

    f1 = f1_score(
        y_test, y_pred, average='weighted'
    )

    print("Accuracy:", accuracy)
    print("Precision:", precision)
    print("Recall:", recall)
    print("F1 Score:", f1)

    # Classification Report
    print("\nClassification Report:")
    print(classification_report(
        y_test, y_pred,
        target_names=data.target_names
    ))

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)

    print("Confusion Matrix:")
    print(cm)

    # Visualize Confusion Matrix
    plt.figure(figsize=(5, 4))

    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=data.target_names,
        yticklabels=data.target_names
    )

    plt.xlabel("Predicted Values")
    plt.ylabel("Actual Values")
    plt.title(name + " Kernel Confusion Matrix")
    plt.tight_layout()
    plt.show()

    # Store results
    results.append([
        name, accuracy, precision, recall, f1
    ])

# 15. Compare all three kernels
result_df = pd.DataFrame(
    results,
    columns=[
        'Kernel', 'Accuracy', 'Precision',
        'Recall', 'F1 Score'
    ]
)

print("\n========== KERNEL COMPARISON ==========")
print(result_df.round(4).to_string(index=False))

# Comparison Graph
result_df.set_index('Kernel').plot(
    kind='bar',
    figsize=(9, 5)
)

plt.title("Comparison of SVM Kernels")
plt.xlabel("Kernel")
plt.ylabel("Performance Score")
plt.ylim(0, 1.1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 16. Vary C and Gamma hyperparameters
print("\n========== HYPERPARAMETER TUNING ==========")

for c in [0.1, 1, 10]:

    for g in [0.1, 1, 'scale']:
```

```
        model = SVC(
            kernel='rbf',
            C=c,
            gamma=g
        )

        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)

        print(
            "C:", c,
            "Gamma:", g,
            "Accuracy:", round(acc, 4)
        )

# 17. Interpret results
print("\n========== FINAL CONCLUSION ==========")

best_model = result_df.loc[
    result_df['Accuracy'].idxmax()
]

print("Best Kernel:", best_model['Kernel'])
print("Best Accuracy:", round(best_model['Accuracy'], 4))

print("\nSVM classification completed successfully.")
```